DATABRICKS

Data Engineer Associate Certification

CHAPTER 4 STUDY GUIDE

Software Engineering, DevOps & CI/CD Fundamentals

Topics Covered	Exam Tips
• SWE Best Practices • Modularizing PySpark Code • DevOps / DataOps / MLOps • CI/CD Pipeline Fundamentals • Unit Tests (PySpark + pytest) • Integration Tests with SDP & Jobs • Version Control with Git • Vocabulary Builder	• Understand CI vs CD vs CD (Delivery vs Deployment) • Know the Testing Pyramid tiers • Know how Git branching works in Databricks • Understand YAML syntax in CI/CD pipelines • assertDataFrameEqual vs assertSchemaEqual

Prepared for Databricks Certified Data Engineer Associate Exam

1. Introduction to Software Engineering (SWE) Best Practices

To build reliable data pipelines, we must borrow from software engineering best practices. These practices ensure pipelines are efficient, scalable, and maintainable over time.

1.1 Overview of Best Practices

There are six core SWE best practices covered in this chapter:

Best Practice	Purpose
Coding Practices	Write readable, modular, linting-friendly code
Document Code	Inline comments, docstrings for maintainability
Automated Testing	Unit tests + integration tests to validate logic
Version Control & Code Review	Git tracking + PR reviews to catch bugs early
CI/CD	Automate build, test, and deploy pipeline
Isolated Environments	DEV / STAGE / PROD separation

1.2 Coding Practices

Code Readability: Write code that is easy to understand, navigate, and maintain. Clear code reduces confusion and minimises errors.

Naming Conventions: Use descriptive, consistent names for variables, functions, and classes. Self-explanatory names enhance collaboration.

Modular Design: Break software into smaller, reusable components (functions). This makes the code easier to test, maintain, and scale.

Code Linting: Linting tools automatically analyse code for potential errors, inconsistencies, and style violations. Examples: flake8, black (Python). Linting is outside the scope of this course but is recommended.

> **KEY POINT**
> Linting tools enforce readability rules automatically. Use them in your CI pipeline to maintain code quality without manual review effort.

1.3 Document Code

Good documentation improves:

- **Code Maintainability:** Clear docs help developers understand purpose and functionality, making updates easier and faster.

- **Collaboration:** Well-documented code lets team members get up to speed fast, reducing misunderstandings.
- **Knowledge Transfer:** Documenting code preserves key design decisions, ensuring smooth transitions when team members change.

Best practices for documentation in PySpark:

- Add module-level docstrings explaining what each file does
- Use function docstrings: parameters, return types, and example usage
- Add inline comments for complex or non-obvious logic

1.4 Automated Testing

Testing is an extremely critical component of software development. Two key categories:

Unit Tests	Integration Tests
Test a single function/component in isolation	Test how different components work together
Fast to run, low cost, high coverage	Slower, but test realistic end-to-end flows
Example: test add_new_col() function	Example: test full Bronze-to-Silver pipeline
Catch logic errors at the source	Catch integration bugs between components

1.5 Version Control & Code Review

Version Control (Git): Tools like Git are essential for tracking changes, collaborating with team members, and maintaining a history of the codebase. Allows rollback, multi-version management, and conflict resolution.

Code Review: When combined with version control, code reviews catch bugs early, improve quality, and ensure consistency in coding standards. They foster collaboration and knowledge sharing within the team.

1.6 CI/CD

CI/CD stands for Continuous Integration and Continuous Deployment/Delivery. It is the main focus of this chapter.

- CI: Developers regularly commit, build, test, and release code to a shared repository.
- CD: Automates the release of code to production after passing automated tests.

1.7 Isolated Environments

Never modify code directly on production. Use separate environments:

Environment	Purpose	Databricks Implementation

DEV	Development and unit testing	Workspace or Catalog (small static subset of prod data)
STAGE	Realistic testing and validation	Workspace or Catalog (larger subset, scrubbed prod data)
PROD	Live, fully operational	Workspace or Catalog (real user data, high security)

In Databricks, you can isolate environments via:

- Multiple Workspaces — one per environment (DEV, STG, PRD)
- Multiple Unity Catalog Catalogs — one per environment within a single Workspace

DATABRICKS ADVANTAGE
Unity Catalog provides built-in lineage, security, and monitoring for all environments without needing third-party tools.

1.8 Software Engineering with Databricks – Tools Overview

Tool / Feature	SWE Use Case	Notes
Notebooks / Files (SQL, Python, Scala)	Develop code and run unit tests	Supported in Workspace
Git Folders	Version control	Significantly improves workflow
Unity Catalog	Isolate environments, manage data access	Single Workspace or multi-Workspace
CI/CD Pipelines (Deployment Tools)	Automated test, deploy	Integrates with GitHub Actions, Azure DevOps, etc.

2. Introduction to Modularising PySpark Code

Modularising code means breaking your PySpark logic into smaller, well-named functions instead of writing everything in one large block or notebook cell.

2.1 Non-Modularised Code – Problems

PROBLEM

Everything is in one block, making it harder to modify or test specific parts. Code duplication is common — the same operation is repeated in multiple places.

Example of non-modularised code:

```
# Load data
df = (spark
      .read
      .csv('health.csv', header=True, inferSchema=True))

# Create column
df = (df
      .withColumn('NewColumn',
        when(col('Column') == 0, 'Normal')
        .otherwise('Unknown')))
```

2.2 Modularised Code – The Solution

Refactor repeated logic into named functions:

```
def load_data(file_path):
    return (spark
            .read
            .csv(file_path, header=True, inferSchema=True))

def add_new_col(df, new, s_col):
    return (df
            .withColumn(new,
              when(col(s_col) == 0, 'Normal')
              .otherwise('Unknown')))
```

2.3 Benefits of Modularising Code

Benefit	Why It Matters
Easier Maintenance	Update specific functions without changing the whole script

Reusability	Reuse load_data() or add_new_col() across different projects
Better Testing	Write unit tests for each individual function (see Section 5)
Readability	Named functions make the pipeline self-documenting
Scalability	Easier to add new transformations without breaking existing code

EXAM TIP

Modularisation is the foundation for unit testing. Without individual functions, you cannot write targeted unit tests. Expect exam questions linking modular design to testability.

3. DevOps Fundamentals

DevOps is a culture and set of practices that combines Software Engineering Best Practices with IT Operations to deliver software more rapidly, efficiently, and with higher quality.

3.1 What is DevOps?

Definition: DevOps fosters collaboration between development and operations teams to automate pipelines, streamline processes, and ensure continuous delivery.

Key benefits of DevOps:

- Faster deployment cycles
- Improved collaboration between teams
- Enhanced system reliability
- Better scalability and efficiency

3.2 The DevOps Lifecycle (8 Stages)

Stage	What Happens	Why It Matters
Plan	Define goals, gather requirements	Aligns the team on what to build
Code	Write the application source code	Building the features & logic
Build	Compile code into executable files	Resolves all dependencies
Test	Run automated tests	Catches bugs before release
Release	Package the application for rollout	Ensures it is production-ready
Deploy	Push to production environment	Makes the app available to users
Operate	Monitor resources, handle issues	Keeps things running smoothly
Monitor	Track performance, gather feedback	Enables continuous improvement

KEY INSIGHT

DevOps is a continuous infinity loop — you keep iterating through these stages as the project needs fixes, updates, or features. There is no 'done'.

3.3 DevOps vs DataOps vs MLOps

DevOps principles can be applied to different technical domains:

Domain	Applies DevOps To	Scope in This Course
DevOps	Software development & IT operations	Core focus — CI/CD, testing, versioning
DataOps	Data engineering pipelines	Covered — automate data pipeline processes
MLOps	Machine learning model lifecycle	Outside scope of this course

DataOps (DevOps for Data Engineering)

DataOps is essentially DevOps applied to data pipelines. Key activities:

- Automate CI/CD for data pipelines
- Centralise data discovery, management, and governance (Unity Catalog)
- Establish traceable data lineage and monitoring
- Enhance collaboration across data teams
- Monitor data quality throughout the pipeline

REMEMBER

DataOps = DevOps principles applied to Data Engineering. Think: 'How can automation, collaboration, and testing improve my data pipeline?'

4. The Role of CI/CD in DevOps

CI/CD is a key subset of DevOps practices that focuses on automating code integration, testing, and delivery pipelines, including DataOps pipelines.

4.1 CI/CD Overview

Concept	Description
CI (Continuous Integration)	Developers regularly merge code into a shared repo. Automated tests run on every merge.
CD (Continuous Delivery)	Code is automatically pushed to staging/pre-prod. Final push to production is a manual approval.
CD (Continuous Deployment)	Fully automated — every change passing tests goes immediately to production. No manual step.

EXAM TIP – TWO TYPES OF CD!
Continuous DELIVERY = automated to stage, manual to prod. Continuous DEPLOYMENT = fully automated to prod. These are commonly confused in exams.

4.2 Continuous Integration (CI) — High Level

CI involves regularly merging code changes from multiple contributors into a central repository and running automated tests to ensure code quality. Tests that fail do not make it into the source code.

Four key benefits of CI:

1. Early Detection of Issues — bugs and conflicts are caught early, when they are cheapest to fix.
2. Faster Development Cycle — frequent integration speeds up delivery of new features.
3. Improved Collaboration and Code Quality — regular integration leads to cleaner, more modular code.
4. Automated Testing and Validation — tests run automatically with each integration.

CI depends on two organisational decisions:

- **Branching Strategy:** How the team organises branches (feature branches, trunk-based, GitFlow, etc.). This affects how and when tests run.
- **Testing Strategy:** Which tests run at which stage and how failures are handled.

4.3 The Testing Pyramid

Within CI/CD there are three tiers of testing. The pyramid shows their relative speed, cost, and scope:

Test Tier	What It Tests	Speed / Cost / Example
Unit Tests (Base)	Individual functions in isolation	Fast, cheap, automated. Ex: test add_new_col()
Integration Tests (Middle)	Interaction between components/systems	Medium speed/cost. Ex: Notebooks + SDP + Jobs
System Tests (Top)	Entire application in a real-world scenario	Slow, expensive. Ex: end-to-end data pipeline in a Job

KEY RULE
Unit tests form the widest base — they are the most numerous because they are fast and inexpensive. System tests are the fewest because they are slow and costly.

4.4 Continuous Delivery vs Continuous Deployment

Continuous Delivery (CD)	Continuous Deployment (CD)
Automated push to staging/pre-production	Fully automated — no human approval step
Manual approval required to deploy to production	Every passing change is immediately live in production
Develop → Build → Test → Version Control → Deploy to Stage → MANUAL → Deploy to Prod	Develop → Build → Test → Version Control → Deploy to Stage → AUTO → Deploy to Prod
Gives the team final control over production releases	Requires well thought-out tests to ensure safety

4.5 High-Level CI/CD Workflow Summary

The full CI/CD pipeline for a Databricks data project:

CI (Continuous Integration)	CD (Continuous Delivery / Deployment)
1. Developer writes code (Notebook/File)	5. Version Control merge triggers deploy
2. Build and dependency check	6. Deploy to Stage environment automatically
3. Unit tests run (pytest + pyspark.testing)	7. Integration/System tests in Stage
4. Push to version control (Git/Databricks Folders)	8. Manual approval (Delivery) or Auto (Deployment) to Prod

5. Planning the Project — CI/CD Pipeline Setup

In this section we plan a data visualisation project using health data, structured across three environments and managed through a CI/CD pipeline.

5.1 Project Requirements

Deliverable: Visualise Health Data through a data pipeline.

Tasks:

5. Ingest daily incremental CSV files into a Bronze table
6. Create a clean Silver table
7. Create Gold tables to share with consumers

Databricks assets used: Workspace, Notebooks, Spark Declarative Pipelines (SDP), Lakeflow Jobs, Compute.

5.2 Setting Up Data Environments

Each environment should have data suited to its purpose:

Environment	Data Characteristics	Purpose
Dev	Small static subset of prod data. Can be anonymised or synthetic.	Rapid development and unit testing. Ensures privacy.
Stage	Mirrors production structure and volume. Sensitive data scrubbed.	Realistic testing and validation before prod.
Prod	Live, fully operational. Contains real user data. Continuously updated.	Final production operations. High security required.

5.3 Isolating Environments in Databricks

Two main approaches to environment isolation:

Approach 1: Multiple Workspaces

- Create a separate Databricks Workspace for DEV, STG, and PRD
- Each Workspace connects to its own cloud storage
- Managed through Unity Catalog at the Workspace level

Approach 2: Unity Catalog Isolation (Single Workspace)

- Create a Unity Catalog Metastore with separate catalogs for dev, stage, and prod
- Dev catalog contains dev data, stage catalog contains stage data, prod catalog contains production data
- Unity Catalog Access Control restricts developers to only the data they need for their environment

5.4 Course Project Architecture

The project architecture used in this course:

- Three Unity Catalog catalogs: Dev (small subset), Stage (larger subset), Prod (daily CSV files)
- Workflow: Unit Tests → Spark Declarative Pipeline → Data Visualisation
- Lakeflow Jobs execute the pipeline: unit tests as tasks, SDP pipeline, final visualisations
- Test through Dev, then Stage, then deploy to Prod once validated

6. Introduction to Unit Tests for PySpark

Unit tests verify that individual functions work correctly. In PySpark, you write unit tests using pyspark.testing.utils and the pytest framework.

6.1 Unit Test Benefits

Benefit	Detail
Test One Thing at a Time	Focus on one function with a small amount of data — easy to isolate problems
Catch Bugs Early	Catch bugs before deploying to production — saves time and reduces errors
Easier Refactoring	Run tests after changing code to ensure nothing broke
Simpler Debugging	Pinpoints exactly where in the code things go wrong

6.2 pyspark.testing.utils — Key Functions

pyspark.testing.utils provides helper functions to make unit testing in PySpark easier:

Function	Purpose
assertDataFrameEqual(actual, expected, ...)	Checks equality between two DataFrames (data + schema). Raises AssertionError if they differ.
assertSchemaEqual(actual, expected)	Checks equality between two DataFrame schemas only. Useful when only structure matters.

6.3 Step-by-Step Unit Test Example

Given this function to test:

```
from pyspark.sql.functions import col, when

def add_new_col(df, new, s_col):
    return (df
            .withColumn(new,
              when(col(s_col) == 0, 'Normal')
              .otherwise('Unknown')))
```

Step 1: Define the expected output:

original (s_col)	desired (new column)
0	Normal
1	Unknown
-1	Unknown
null	Unknown

Step 2: Write the unit test:

```
def test_add_new_col():
    # Create sample DataFrame
    data    = [(0,), (1,), (-1,), (None,)]
    columns = ['value']
    df      = spark.createDataFrame(data, columns)

    # Execute the function
    actual_df = add_new_col(df, 'new_value', 'value')

    # Define expected result
    expected_data = [(0,'Normal'), (1,'Unknown'),
                     (-1,'Unknown'), (None,'Unknown')]
    expected_df   = spark.createDataFrame(expected_data,
                                          ['value','new_value'])

    # Assert equality
    assertDataFrameEqual(actual_df, expected_df)
```

GOAL

Compare the ACTUAL result of the function with a DEFINED EXPECTED result. If they match, the test passes. If not, an AssertionError is raised with details.

6.4 The pytest Framework

pytest is the most popular testing framework for Python, making it easy to write simple and scalable test cases.

Feature	Description
Simple Syntax	Define functions starting with test_ — no boilerplate setup needed
Assertions	Use Python built-in assert or PySpark's assertDataFrameEqual for detailed error messages
Automatic Discovery	pytest automatically finds and runs all functions starting with test_

Rich Ecosystem	Plugins for coverage reports, parallel test execution, and integration with other tools

Running tests with pytest:

```
# Run all tests in a file
pytest test_my_functions.py

# Run a specific test function
pytest test_my_functions.py::test_add_new_col

# Run with verbose output
pytest -v test_my_functions.py
```

NOTE

This course provides a simple introduction to pytest. Many testing frameworks exist — select the one that best meets your organisation's needs.

7. Executing Integration Tests with SDP and Jobs

Integration tests check how different components interact — for example, whether a Spark Declarative Pipeline (SDP) correctly processes data end-to-end when connected to notebooks and Lakeflow Jobs.

7.1 Two Ways to Run Integration Tests in Databricks

Method	How It Works
Lakeflow Spark Declarative Pipelines (SDP)	Use SDP expectations to validate table results within the pipeline. This is the primary method used in this course.
Databricks Lakeflow Jobs	Implement integration tests as a Databricks Job with multiple notebook tasks (for non-SDP code).

7.2 SDP Expectations for Integration Testing

Within a Spark Declarative Pipeline, you can define expectations on tables. These expectations act as integration test assertions:

```
@dlt.expect_all_or_drop({
    'valid_patient_id': 'patient_id IS NOT NULL',
    'valid_age':        'age BETWEEN 0 AND 130'
})
def silver_health():
    return dlt.read('bronze_health')
```

Types of SDP expectations:

- **expect:** Tracks constraint violations but keeps all rows.
- **expect_or_drop:** Drops rows that violate the constraint.
- **expect_or_fail:** Fails the pipeline if any row violates the constraint (most strict — acts as a hard integration test gate).

INTEGRATION TEST GATE

Using expect_or_fail in staging environment acts as an integration test gate — if expectations fail, the pipeline stops and deployment is blocked until the issue is fixed.

7.3 Integration Tests via Lakeflow Jobs

For non-SDP pipelines, use Databricks Lakeflow Jobs with multiple tasks:

8. Task 1: Run unit tests (pytest on PySpark functions)
9. Task 2: Execute the pipeline notebook

10. Task 3: Run integration validation notebook (check row counts, schema, data quality)
11. Task 4: (Optional) Run a downstream notebook that reads from output tables

Each task can be configured with conditional dependencies — if one task fails, downstream tasks are skipped or the job stops.

8. Version Control with Git — Easy Explanation

This section explains Git from the ground up, resolving common doubts, with a focus on how it applies to Databricks workflows.

8.1 What is Git and Why Do We Use It?

Think of Git as a time machine for your code. Every time you make a meaningful change, you save a snapshot (called a commit). If something goes wrong later, you can go back to any previous snapshot.

Without Git:

- You have one copy of the code. If you break it, it is broken.
- Two people editing the same file = someone's work gets overwritten.
- No history of who changed what, or why.

With Git:

- Every change is tracked with a history of who made it and when.
- Multiple people can work on different features simultaneously without stepping on each other.
- You can roll back any change instantly.
- Easily see the differences between any two versions of a file.

SIMPLE ANALOGY

Git is like Google Docs version history for your code — but far more powerful, because you can work on separate 'copies' (branches) and merge them back together later.

8.2 Core Git Concepts — Explained Simply

Term	Simple Explanation	Technical Definition
Repository (Repo)	A folder that Git is watching — it tracks every change inside	A directory containing your project plus a hidden .git folder with the full history
Commit	A saved snapshot of your code at a point in time. Like pressing 'Save' with a message.	A hash-identified record containing changed files, author, timestamp, and parent commit
Branch	A separate workspace where you can make changes without affecting the main code	A lightweight movable pointer to a specific commit

Main / Master	The official, stable version of the code that is deployed to production	The default branch in a repository; the source of truth
Feature Branch	A temporary branch you create to work on one feature or fix	A branch created from main to develop a specific feature in isolation
Merge	Combining your feature branch changes back into the main branch	Integrating changes from one branch into another
Pull Request (PR)	Asking teammates to review your changes before merging into main	A formal code review request; triggers automated tests in CI
Clone	Download a copy of a repository to your computer or Databricks	Creates a local copy of a remote repository
Push	Send your local commits to the remote repository (GitHub/Azure DevOps)	Uploads local branch commits to a remote
Pull	Download the latest changes from the remote repository to your local copy	Fetches and merges remote changes into your local branch
Conflict	Two people edited the same line — Git does not know which one to keep	A state where Git cannot automatically merge changes

8.3 A Typical Git Workflow (Day-to-Day)

Here is what a typical Git-based development day looks like:

12. Pull the latest code from the main branch (get up to date)
13. Create a new feature branch (e.g. feature/add-silver-transform)
14. Write and test your code in Databricks notebooks or files
15. Commit your changes with a meaningful message (e.g. 'Add silver transformation for age column')
16. Push your branch to the remote repository (GitHub / Azure DevOps / Bitbucket)
17. Open a Pull Request (PR) for code review
18. CI pipeline runs automatically (unit tests, linting, integration tests)
19. Teammates review the PR, suggest changes, or approve
20. Merge the PR into the main branch
21. CD pipeline triggers — deploys to Stage, then Prod

WHY PRS MATTER

A Pull Request is the gate between your feature branch and the main branch. It enforces code review AND triggers CI tests. Nothing goes to main without passing both. This is how CI/CD and Git work together.

8.4 Branching Strategies

Different teams use different branching models. The most common:

Strategy	How It Works / When to Use
Trunk-Based Development	All developers commit directly to main (or very short-lived branches). Works well for small teams with strong test coverage and fast CI.
Feature Branch Workflow	Each new feature or fix gets its own branch. Merged via PR. Most common approach in teams of any size.
GitFlow	Has long-lived branches: main, develop, release, hotfix. More structured, suited for teams with scheduled releases. More overhead.
Environment Branches (DEV/STG/PROD)	Separate branches for each environment. A merge from dev to staging triggers deployment to staging. Merge from staging to main deploys to prod.

EXAM NOTE
The course focuses on Continuous Integration — understanding how branching strategy determines when tests run and when code is merged is key to exam questions.

8.5 Git in Databricks — Git Folders

Databricks has built-in Git integration called Git Folders (also known as Databricks Repos in older versions).

Key things you can do in Databricks with Git Folders:

- Connect a Databricks Folder directly to a Git repository (GitHub, Azure DevOps, Bitbucket, GitLab)
- Pull the latest code from the remote repo into your Databricks environment
- Commit and push changes from within the Databricks UI
- Switch between branches from within Databricks
- View diffs and commit history

Typical Databricks Git Folder setup for CI/CD:

```
Databricks Workspace
  └── Git Folders
        └── my_data_pipeline_project/   ← Connected to GitHub repo
              ├── notebooks/
              │     ├── bronze_ingestion.py
              │     ├── silver_transform.py
              │     └── gold_aggregation.py
              ├── tests/
              │     └── test_silver_transform.py
              └── pipeline.yml           ← CI/CD pipeline config (YAML)
```

8.6 Common Git Commands Reference

Command	What It Does	Example
git clone <url>	Download a repo to your machine	git clone https://github.com/org/repo.git
git checkout -b <branch>	Create and switch to a new branch	git checkout -b feature/silver-transform
git status	See which files have changed	git status
git add <file>	Stage changes for commit	git add notebooks/silver_transform.py
git commit -m 'message'	Save staged changes as a commit	git commit -m 'Add age validation logic'
git push origin <branch>	Push your branch to the remote	git push origin feature/silver-transform
git pull origin main	Pull latest main into your branch	git pull origin main
git merge <branch>	Merge another branch into current	git merge feature/silver-transform
git log	View commit history	git log --oneline
git diff	See line-by-line changes	git diff notebooks/silver_transform.py

8.7 Resolving Merge Conflicts

A merge conflict happens when two people change the same line of code in different ways. Git marks the conflict in the file:

```
<<<<<<< HEAD (your branch)
```

```
df = df.withColumn('age_group', when(col('age') < 18,
'Minor').otherwise('Adult'))
=======
df = df.withColumn('age_group', when(col('age') < 21,
'Minor').otherwise('Adult'))
>>>>>>> feature/silver-transform
```

To resolve: decide which version to keep (or write a combined version), remove the conflict markers, then commit the resolution. Communication with teammates is the real solution.

9. YAML in CI/CD Pipelines

YAML (Yet Another Markup Language) is the format used to define CI/CD pipeline configuration files. Understanding YAML is essential for working with GitHub Actions, Azure DevOps Pipelines, and Databricks deployment tooling.

9.1 What is YAML?

YAML = Yet Another Markup Language. It is a human-readable data serialisation format used for configuration files. CI/CD tools read YAML files to know what steps to execute, when to run them, and on what conditions.

Key YAML rules:

- Indentation matters — use spaces, never tabs (2 spaces is standard)
- Key-value pairs use a colon: name: value
- Lists use a dash: - item
- Strings do not need quotes unless they contain special characters
- Comments start with #

9.2 Simple YAML Example — GitHub Actions CI Pipeline

```yaml
name: Databricks CI Pipeline

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]

jobs:
  run-unit-tests:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v3

      - name: Set up Python
        uses: actions/setup-python@v4
        with:
          python-version: '3.10'

      - name: Install dependencies
        run: pip install pytest pyspark

      - name: Run unit tests
        run: pytest tests/ -v
```

Breaking down the YAML above:

YAML Key	What It Means
name:	A human-readable name for this pipeline workflow
on: push / pull_request	Triggers — when does this pipeline run? On a push or PR to main/develop
jobs:	The top-level container for all jobs in this workflow
runs-on:	The type of runner/machine the job executes on
steps:	The ordered list of actions to perform in this job
uses:	Reference to a pre-built action from GitHub Marketplace
run:	Execute a shell command directly

WHY YAML MATTERS FOR DATABRICKS

Databricks CI/CD pipelines (GitHub Actions, Azure DevOps) are all configured via YAML. Knowing how to read and write a YAML pipeline is essential for setting up automated deployments.

10. Vocabulary Builder — Key Terms Explained Simply

This section explains important new terms used throughout Chapter 4, with simple real-world analogies.

Word	Simple Meaning	In Context of Chapter 4
Isolating	Keeping things separate so they do not interfere with each other.	Isolating environments (DEV, STAGE, PROD) means changes in DEV cannot accidentally break PROD. Like separate labs in a hospital so infection does not spread.
Readable	Easy to understand and follow, even for someone who did not write it.	Code readability means another developer (or future-you) can read your code and quickly understand what it does, without needing the original author to explain it.
Reliability	Consistently doing what is expected, without unexpected failures.	A reliable data pipeline delivers the correct output every time it runs — not just sometimes. Automated testing is what achieves reliability.
Arbitrary	Chosen randomly or without a specific reason — not following a rule.	In data engineering: 'arbitrary logic' means a rule that was written without a specific standard, which can vary unpredictably. We want rules to be well-defined, not arbitrary.
Encapsulating	Wrapping something up inside a container so that the internal details are hidden.	Encapsulating logic in a function (def add_new_col()) means you only need to know what the function does (its input and output), not how it does it internally.
YAML	A simple human-readable file format for configuration, using indentation and key-value pairs.	CI/CD pipelines are written in YAML files (like .github/workflows/ci.yml). The file tells GitHub Actions or Azure DevOps what commands to run and when to run them.
Modular	Made up of separate, self-contained pieces (modules) that can be used independently.	Modular PySpark code means breaking your notebook into functions. Each function handles one task. You can test, update, or reuse each function independently.
Pipeline	A series of connected steps where the output of one step becomes the input of the next.	A data pipeline: Ingest CSV (Bronze) → Clean data (Silver) → Aggregate (Gold). A CI/CD pipeline: Code → Build → Test → Deploy.

Deployment	The act of releasing software or a data pipeline to an environment where it can be used.	Deploying to production means your new code is live and running for real users/consumers. CI/CD automates this deployment process.
Continuous	Happening regularly and without stopping — not once-off, but ongoing.	Continuous Integration means developers integrate their code continuously (many times a day), not in large batches. Continuous Deployment means every approved change is deployed without delay.
Integration	The process of bringing different parts together and making them work as a whole.	Integration tests check that multiple components (notebook + SDP + Job) work correctly together — not just in isolation like unit tests do.
Refactoring	Improving the internal structure of code without changing its external behaviour.	Refactoring non-modular code into functions makes it cleaner and testable, without changing what the code actually does.

10.1 Git-Specific Vocabulary

Git Term	Plain English Meaning
Repository	A project folder being tracked by Git — all the code and its complete history
Commit	A saved checkpoint with a message describing what changed and why
Branch	A parallel workspace to develop a feature without touching the main code
Merge	Combining a feature branch back into the main branch
Pull Request	A formal request to review and merge your code — triggers CI tests
Clone	Downloading a copy of a remote repository to your local machine
Push	Uploading your local commits to the remote repository (GitHub, etc.)
Pull	Downloading the latest changes from remote into your local copy
Conflict	Two people edited the same line — Git cannot auto-decide which to keep

HEAD	The current state of your active branch — where you are right now in Git history
Remote	The cloud-hosted version of the repository (GitHub, Azure DevOps, GitLab, etc.)
Tag	A marker on a specific commit — often used to label releases (v1.0, v2.3)

11. Exam Quick Reference & Practice Questions

11.1 Chapter 4 — Must Know Summary

SWE BEST PRACTICES

6 practices: Coding Practices, Document Code, Automated Testing, Version Control & Code Review, CI/CD, Isolated Environments.

TESTING PYRAMID

Bottom = Unit Tests (fast, cheap, test functions in isolation). Middle = Integration Tests (components together). Top = System Tests (full end-to-end, slowest).

CI VS CD VS CD

CI = merge + test continuously. Continuous DELIVERY = auto push to stage, MANUAL push to prod. Continuous DEPLOYMENT = fully automated to prod.

ASSERTDATAFRAMEEQUAL VS ASSERTSCHEMAEQUAL

assertDataFrameEqual checks data AND schema. assertSchemaEqual checks schema ONLY. Both are in pyspark.testing.utils.

GIT IN DATABRICKS

Use Databricks Git Folders to connect notebooks to GitHub/Azure DevOps. Supports clone, pull, push, branch switch from within the UI.

11.2 Practice Questions

Q1. What is the primary benefit of modularising PySpark code into functions?

ANSWER

It makes the code easier to maintain, reuse, and test. Individual functions can be unit-tested in isolation, which is not possible with a single monolithic code block.

Q2. Which testing tier tests individual PySpark functions in isolation?

ANSWER

Unit Tests — the base of the Testing Pyramid. They are the fastest, cheapest, and most numerous tests in a CI/CD pipeline.

Q3. What is the difference between Continuous Delivery and Continuous Deployment?

ANSWER

Continuous Delivery automates deployment to stage but requires a manual approval to push to production. Continuous Deployment is fully automated — every passing change goes directly to production without human intervention.

Q4. In Databricks, what are the two methods for isolating environments?

ANSWER

(1) Multiple Workspaces — one per environment (DEV, STG, PRD). (2) Unity Catalog isolation — create separate catalogs (dev, stage, prod) within a single Workspace. Unity Catalog Access Control restricts developer access per environment.

Q5. What does YAML stand for and where is it used in a Databricks CI/CD context?

ANSWER

YAML = Yet Another Markup Language. It is used to define CI/CD pipeline configuration files (e.g., GitHub Actions workflows in .github/workflows/ci.yml). The YAML file specifies triggers, jobs, and steps — including running pytest unit tests and deploying to Databricks environments.

Q6. What is the purpose of assertDataFrameEqual() in unit testing?

ANSWER

It compares an actual PySpark DataFrame (output of your function) with an expected DataFrame you define. If they are equal, the test passes. If not, an AssertionError is raised with details on what differs.

Q7. In the context of Git, what is a Pull Request (PR)?

ANSWER

A Pull Request is a formal request to merge a feature branch into the main branch. It triggers automated CI tests and allows teammates to review the code changes before merging. It is the main quality gate between feature development and the production codebase.

Q8. How do SDP expectations function as integration tests in Databricks?

ANSWER

SDP (Spark Declarative Pipeline) expectations define data quality constraints on pipeline tables. Using expect_or_fail, any row violating the constraint fails the entire pipeline — acting as an integration test gate that prevents bad data from advancing to the next environment.